

Data Types : Complete Notes

(handwritten-style summary of Session 4 · Lists, Session 5 · Tuples-Sets-Dicts, Session 6 · Functions - folder 02_Python_Data_Types)

Q0. What does Module 02 give you?

Answer : The **containers** - list, tuple, set, dict - plus **functions**. Choosing the right container decides whether your code is a clean pipeline or a tangle of loops.

Logic - Study order: 01 Lists → 02 Tuples/Sets/Dicts → 03 Functions → Tasks → Interview questions.

Session 4 · Lists

Q1. What is a List? List vs Array?

Answer : A list is a data type that stores **many items under one name** - technically a **dynamic array** that grows and shrinks on the fly.

Logic - You COULD write 100 separate variables for 100 students - a list lets one name hold the whole collection.

Array (C / Java)	Python List
Fixed size - planned before filling	Dynamic - grows / shrinks automatically
Homogeneous (one type)	Heterogeneous - mixed types allowed
Very fast, very compact	A little slower, eats more memory
Values sit side by side	Stores references to objects

Q2. How is a list stored in memory?

Answer : As a contiguous block of **references (pointers)** to objects that live elsewhere in memory.

memory.py

```
1 L = [1, 2, 3]
2 print(id(L))      # address of the list itself
3 print(id(L[0]))   # SAME id as id(1) - small ints are cached
4 print(id(1))
```

Logic - The list does not copy the numbers inside itself - `L[0]` points to the very same integer object as the literal `1`.

Q3. Characteristics of a List - the 8 rules?

Answer : Ordered · Mutable · Heterogeneous · Duplicates OK · Dynamic · Nestable · Index access · Any object allowed.

Logic - Order is part of the VALUE: `[1,2,3] == [3,2,1]` → False . And a list can even hold functions: `[print, type, input]` is legal.

{ "Mutable" = the list can be changed IN PLACE (append, edit, delete). Strings & tuples cannot - that difference shows up everywhere. **Hint** :

Q4. Creating a list - every legal way?

```
create.py

1 [] # empty
2 [1, 2, 3, 4, 5] # 1D homogeneous
3 [1, 2, 3, [4, 5]] # 2D - a list inside a list
4 [[1,2],[3,4]],[[5,6],[7,8]] # 3D
5 [1, True, 5.6, 5+6j, 'Hello'] # heterogeneous
6 list('hello') # type conversion -> ['h','e','l','l','o']
7 list(range(5)) # -> [0,1,2,3,4]
```

`[]` is the fast literal. `list()` is the converter you call on any iterable (string, tuple, range...).

Q5. Indexing & Slicing?

Answer : Indexing grabs ONE item by position; slicing grabs a RANGE and returns a new list.

```
access.py

1 L = [1, 2, 3, 4, 5, 6]
2 L[0] # 1 (left, starts at 0)
3 L[-1] # 6 (right, starts at -1)
4 L[1:4] # [2,3,4] stop is EXCLUSIVE
5 L[::2] # [1,3,5] every 2nd
6 L[::-1] # [6,5,4,3,2,1] REVERSE (famous one-liner)
7 L[9] # IndexError: list index out of range
```

Q6. Adding items - append vs extend vs insert?

`append` adds ONE box · `extend` dumps the CONTENTS of a box · `insert` drops a value at a position (shifts right).

```

add.py

1 L = [1,2,3,4,5]
2 L.append([6,7,8])    # -> [1,2,3,4,5,[6,7,8]]  NESTED!
3 L.extend([6,7,8])    # -> [1,2,3,4,5,6,7,8]    flat merge
4 L.extend('delhi')    # strings split into chars! ...'d','e','l','h','i'
5 L.insert(1, 100)     # -> [...,100,...] at position 1

```

Q7. Editing items - in place?

```

edit.py

1 L = [1,2,3,4,5]
2 L[-1] = 500          # edit by (negative) index
3 L[1:4] = [200,300,400] # slice assignment
4 print(L)             # [1,200,300,400,500]

```

Logic - Slice assignment can even change the LENGTH - `L[1:3] = [7,8,9,10]` grows the list, `L[1:4] = []` shrinks it. Strings/tuples can't do this.

Q8. Deleting items - the four tools?

```

delete.py

1 del L[-1]            # by index          (nothing returned)
2 del L[1:3]           # by slice
3 L.remove(3)          # by VALUE - first occurrence (ValueError if missing)
4 L.pop()              # removes AND returns the last item (LIFO)
5 L.pop(0)             # removes AND returns the item at index 0
6 L.clear()            # empty the whole list -> []
7 del L               # delete the VARIABLE itself (NameError later)

```

`pop()` gives the value BACK to you; `del` / `remove` just delete. Use `pop` when you still need that item.

⚠ important - `remove()` uses the VALUE, `del` uses the INDEX. Mixing them up is the #1 beginner bug.

Q9. Operations on lists - +, *, in, loop, ==?

```
ops.py

1 [1,2] + [3,4]          # [1,2,3,4]  concatenation - NEW list
2 [1,2] * 3              # [1,2,1,2,1,2] repetition
3 5 not in [1,2,3,4,5]   # False - membership works at any depth
4 [5,6] in [1,2,3,4,[5,6]] # True!
5 for i in L: print(i)   # walk items in order
6 [1,2,3] == [3,2,1]     # False - ORDER is part of the value
```

Q10. List functions - the toolkit?

Tool	What it does	New or in place?
len / min / max	size · smallest · largest	new result
sorted(L)	sorted copy (reverse=True for desc)	new list - L untouched
L.sort()	sorts in place	mutates, returns None
L.count(x)	how many times x appears	result
L.index(x)	position of FIRST x (ValueError if missing)	result
L.reverse()	reverses permanently	mutates
L.copy()	a separate copy	new list (shallow!)

```
sort_vs_sorted.py

1 L = [2,1,5,7,0]
2 print(sorted(L))   # [0,1,2,5,7]  L still [2,1,5,7,0]
3 L.sort()           # L becomes [0,1,2,5,7] (returns None)
4 # aliasing trap:
5 b = L               # b IS the same list - edit L, b changes too
6 b = L.copy()        # separate copy - fully independent
```

⚠ important - `copy()` is SHALLOW - nested lists are still shared. Use `copy.deepcopy` for full independence.

Q11. List Comprehension?

```
newlist = [ expression for item in iterable if condition ]
```

3 parts: the compute, the loop, the filter. Faster + fewer lines than append-loops.

comprehension.py

```
1 [i for i in range(1,11)]           # 1..10
2 [i**2 for i in [1,2,3,4,5]]        # squares
3 [i for i in range(1,51) if i%5==0] # multiples of 5
4 [x for x in languages if x.startswith('p')]
5 [f for f in fruits if f in basket if f.startswith('a')] # stacked if = AND
6 [i*j for i in L1 for j in L2]       # cartesian product
7 [[i*j for i in range(1,4)] for j in range(1,4)] # 3x3 matrix
```

Read it like the loop it replaces: expression first (where `append()` sat), loop second, filter after - the rightmost `for` is the innermost loop. Hint :

Q12. Two ways to traverse?

traverse.py

```
1 # itemwise - read values only
2 for i in [1,2,3,4]: print(i)
3 # indexwise - need the position OR must edit
4 for i in range(len(L)): print(L[i])
5 # pythonic middle ground
6 for i, val in enumerate(L): print(i, val)
```

Q13. `zip()` - pairing lists?

Answer : `zip(L1, L2)` pairs items position by position into tuples; the shortest list decides the length.



zip.py

```
1 L1 = [1,2,3,4]; L2 = [-1,-2,-3,-4]
2 list(zip(L1, L2))           # [(1,-1),(2,-2),(3,-3),(4,-4)]
3 [i+j for i, j in zip(L1, L2)] # index-wise addition
```

`zip()` returns an ITERATOR - wrap it in `list()` to see it. Its real power: unpacking in a comprehension `for i, j in zip(...)`.

TAGDA · Session 4 homework

- Split a list into odds & evens (two accumulators + `% 2`).
- Take a list from the user: `input().split()` then `map(int, ...)`.
- Merge two lists WITHOUT `+` - use `extend`.
- Replace every 3 with 300 (`index()` + assignment).
- Flatten a 2D list to 1D (row loop + cell loop).
- Remove duplicates - without losing the order (a `seen` set-check).
- Check if a list is in ascending order (`L == sorted(L)`).

Session 5 · Tuples, Sets, Dicts

Q14. What is a tuple?

Answer : A list that **cannot change** - an **immutable** list. Same sequence of items, but edit/append/delete are illegal.

Logic - Characteristics: **Ordered** · **Unchangeable** · **Duplicates allowed**.

```
create_tuple.py

1 t1 = ()                # empty tuple
2 t2 = ('hello',)        # ONE item - trailing comma MANDATORY!
3 t3 = (1,2,3,4)          # homogeneous
4 t4 = (1, 2.5, True, [1,2,3]) # heterogeneous (even a list!)
5 t5 = (1,2,3, (4,5))     # nested tuple
6 t6 = tuple('hello')     # -> ('h','e','l','l','o')
7 # trap: ('hello') is just a str; ('hello',) is the tuple
```

Q15. Accessing - indexing & slicing?

```
access_tuple.py

1 t3[0]      # 1                (positive from left, 0-based)
2 t3[-1]     # 4                (negative from right, -1 = last)
3 t5[-1]     # (4,5)            nested access
4 t5[-1][0]  # 4                chain the brackets
5 t = (1,2,3,4,5)
6 t[-1:-4:-1] # (5,4,3)        reverse slice - result is a TUPLE
```

Q16. Can we edit or delete a tuple?

Answer : No - that is the whole point of immutability.

```
immutable.py

1 t3[0] = 100  # TypeError: tuple does not support item assignment
2 del t5[-1]   # TypeError: tuple doesn't support item deletion
3 del t3       # whole-variable delete IS allowed
4 print(t3)    # NameError later
```

Logic - Immutability = safety: a tuple can live inside a set or be a dict key (a list cannot), its contents can't change under you, and it is faster + smaller.

Q17. Tuple operations & functions?

```
tuple_ops.py

1 (1,2,3) + (4,5,6)      # (1,2,3,4,5,6)
2 (1,2) * 3               # (1,2,1,2,1,2)
3 1 in (1,2,3)           # True
4 len(t) / sum(t) / min(t) / max(t)
5 sorted(t, reverse=True) # returns a LIST, not a tuple!
6 t.count(2)             # 1 (missing -> 0)
7 t.index(2)             # 1 (missing -> ValueError)
```

A tuple has ONLY two methods - count() and index(). Lists carry append/extend/sort/reverse etc. because a tuple cannot mutate at all. **Hint :**

Q18. List vs Tuple - the real differences?

Point	List	Tuple
Syntax	[...]	(...)
Mutability	mutable	immutable
Speed (iteration)	slower	faster ($\approx 1.2\times$, tested on 10^8 items)
Memory	bigger (9120 B)	smaller (8056 B for 1000 ints)
Built-in methods	many (append, sort...)	only count / index
Error prone	more (can be changed by accident)	safer
Usability	when data grows/changes	for fixed records, keys, return values

Logic - Aliasing subtlety: `a = a + (4,)` makes a **brand-new** tuple (a rebinds, b keeps the old). `a.append(4)` would have changed the very object b sees.

Q19. Special tuple syntax - unpacking?

```
unpacking.py

1 a, b, c = (1, 2, 3)      # destructuring
2 a, b = (1,2,3)           # ValueError: too many values to unpack
3 a, b = b, a              # THE pythonic swap - no temp needed
4 a, b, *rest = (1,2,3,4)  # star grabs the rest as a LIST
5 tuple(zip((1,2), (5,6))) # ((1,5),(2,6))
6 def stats(L): return min(L), max(L) # multiple returns = tuple
7 lo, hi = stats([3,1,4])  # unpack the tuple in one line
```

Part 2 · Sets

Q20. What is a set?

Answer : An **unordered** collection of **unique** items, where every element must itself be immutable. The set itself IS mutable (add/remove allowed).

```
create_set.py

1 s = set()          # EMPTY set ({ } is a DICT!)
2 s1 = {1, 2, 3}      # literal
3 s3 = {1, 'hello', 4.5, (1,2,3)} # elements must be HASHABLE
4 s5 = {1,1,2,2,3,3}  # duplicates vanish -> {1,2,3}
5 s6 = {1,2,[3,4]}    # TypeError: unhashable type: 'list'
6 {1,2,3} == {3,2,1} # True - order is irrelevant
```

Q21. Why can't we index or slice a set?

Answer : Because it is **unordered** - there is no "position 0".

```
set_no_index.py

1 s1[0:3]          # TypeError: 'set' object is not subscriptable
2 s1[0] = 100      # TypeError: 'set' object does not support item assignment
```

O(1) membership! Sets use a hash table - "is it there?" is instant, whereas a list scans one by one ($O(n)$). The trade-off for losing order.

Q22. Adding & removing items?

```
set_add_remove.py

1 S = {1,2,3,4}
2 S.add(5)           # one item
3 S.update([6,7,8])  # many items from any iterable
4 S.discard(50)       # gentle - no error if missing
5 S.remove(50)        # strict - ValueError if missing
6 S.pop()             # removes & returns SOME random item
7 S.clear()           # -> set()
8 del S               # delete the whole variable
```

Q23. Set operations - | & - ^?

```
set_ops.py

1 s1 = {1,2,3,4,5}; s2 = {4,5,6,7,8}
2 s1 | s2  # UNION      {1,2,3,4,5,6,7,8}  everything
3 s1 & s2  # INTERSECTION {4,5}           in BOTH
4 s1 - s2  # DIFFERENCE  {1,2,3}          in s1 not s2
5 s1 ^ s2  # SYMMETRIC   {1,2,3,6,7,8}    in one, not both
6 1 not in s1  # membership & iteration work normally
```

This is WHY sets exist - deduplicate, and answer "are they common / different?" in ONE line. Hint :

Q24. Set methods - new vs in-place?

Returns a <i>NEW</i> set	Mutates (<i>*_update</i>)
<code>union()</code>	<code>update()</code>
<code>intersection()</code>	<code>intersection_update()</code>
<code>difference()</code>	<code>difference_update()</code>
<code>symmetric_difference()</code>	<code>symmetric_difference_update()</code>
comparisons: <code>isdisjoint()</code> · <code>issubset()</code> · <code>issuperset()</code> · <code>copy()</code>	

Logic - Plain method = look only (returns result). `*_update` = change the original set. Use `update` when you WANT to modify; plain when you want to keep both sets.

Q25. frozenset & set comprehension?

Answer : A frozenset is the **immutable** set - read operations work, write ones throw. Because it's hashable, frozensets can nest → "2D sets".

```
frozen_comp.py

1 fs = frozenset([1,2,3])
2 fs | frozenset([3,4,5])    # ok - read ops allowed
3 fs.add(6)                  # TypeError - write ops forbidden
4 frozenset([1,2,frozenset([3,4])]) # nesting works!
5 {i**2 for i in range(1,11) if i > 5} # set comprehension
```

Part 3 · Dictionaries

Q26. What is a dictionary?

Answer : A collection of **key** → **value** pairs - a map / associative array (JSON's twin).
Lookup is BY KEY, not by position.

```
create_dict.py

1 d = {} # empty dict
2 d1 = {'name': 'nitish', 'gender': 'male'}
3 d2 = {(1,2,3): 1, 'hello': 'world'} # tuple keys ok
4 s = {'name': 'nitish', 'subjects': {'dsa': 50, 'maths': 67}} # nested = dict
5 d4 = dict([('name','nitish'), ('age',32)]) # from pairs
6 {'name':'a', 'name':'b'} # duplicate key -> LAST wins: {'name':'b'}
7 {'name':'a', [1,2]:'x'} # keys MUST be immutable (hashable)
```

Q27. Reading values - [] vs get()?

```
read_dict.py

1 d = {'name': 'Jack', 'age': 26}
2 d['age'] # 26
3 d['xxx'] # KeyError: 'xxx' CRASH!
4 d.get('age') # 26
5 d.get('xxx') # None safe - no crash
6 d.get('xxx', 0) # 0 custom default
7 s['subjects']['maths'] # nested access chains [ ][ ]
```

⚠ important - [] is fast but throws `KeyError` on a missing key. `get()` never throws. When the key MAY be absent → use `get()`.

Q28. Add, edit & remove entries?

modify_dict.py

```
1 d4['gender'] = 'male'    # insert (key new)
2 d4['weight'] = 72        # insert
3 d4['age'] = 33           # EDIT (key exists) - same syntax!
4 d4.pop('name')          # removes & returns the value
5 d4.popitem()            # removes & returns the LAST pair
6 del d4['age']            # removes silently (KeyError if missing)
7 d4.clear()              # empty the dict
8 s['subjects']['ds'] = 75 # nested add works the same
```

One syntax, two jobs : assigning `d[k] = v` INSERTS if `k` is new and OVERWRITES if it exists. There is no separate "add" method.

Q29. Membership & iteration?

dict_loop.py

```
1 'name' in d              # True - in checks KEYS, never values
2 for i in d: print(i, d[i]) # i = key
3 d.keys()                # keys only
4 d.values()              # values only
5 d.items()               # (key, value) pairs - tuple unpacking!
6 for k, v in d.items(): ...
```

Q30. Dict functions & update()?

dict_funcs.py

```
1 len(d)                  # number of pairs
2 sorted(d, reverse=True) # keys sorted -> a LIST
3 max(d)                  # largest KEY
4 d1 = {1:2, 3:4, 4:5}
5 d2 = {4:7, 6:8}
6 d1.update(d2)           # d1 absorbs d2; shared key 4 -> 7
```

Logic - `items()/keys()/values()` are LIVE views - `list()` them to freeze a snapshot. `update()` merges the second dict INTO the first, later values winning.

Q31. Dictionary comprehension?



dict_comp.py

```
1 {i: i*i for i in range(1,11)}           # n -> n²
2 {k: v*0.62 for k, v in distances.items()} # transform values
3 {i: j for i, j in zip(days, temp_C)}     # pair two lists
4 {k: v for k, v in stock.items() if v > 0} # filter
5 {i: {j: i*j for j in range(1,11)} for i in range(2,5)} # nested tables
```

TAGDA · Session 5 homework

- Convert km distances to miles with a dict comprehension.
- Pair `days` and `temp_C` into a dict with `zip()`.
- Keep only in-stock products (filter with `if value > 0`).
- Build multiplication tables 2-4 as a nested dict comprehension.
- Swap two variables, then delete a dict key - without looking at notes.

Session 6 · Functions

Q32. Anatomy of a function?

```
is_even.py

1 def is_even(num):
2     """input - any integer ; output - 'even'/'odd'""" # docstring
3     if type(num) == int:
4         return 'even' if num % 2 == 0 else 'odd'
5     return 'pagal hai kya?' # non-int input is handled
6 for i in range(1,11): print(is_even(i))
```

Logic - `def name(params):` · indented body · `return` hands a value back. The **docstring** (triple quotes, first line) documents input/output/purpose. Call with `name(args)` .

Q33. Two points of view - and Parameters vs Arguments?

Answer : The author writes the function; the user only passes inputs and gets the output. Parameters are names at the **definition**; arguments are values at the **call**.

```
params_args.py

1 def power(a, b): # a, b = PARAMETERS (definition time)
2     return a ** b
3 power(2, 3) # 2, 3 = ARGUMENTS (call time) -> 8
```

Q34. Types of arguments?

```
arg_types.py

1 def power(a=1, b=1): # DEFAULT arguments
2     return a ** b
3 power() # 1 both defaults used
4 power(2, 3) # 8 POSITIONAL - matched by order
5 power(b=3, a=2) # 8 KEYWORD - matched by name, order free
6 power(2, b=8) # 256 mixing: positionals first
7 power(a=2, 3) # SyntaxError - keyword then positional is illegal
```

Q35. *args & **kwargs?

Answer : The stars collect a variable number of arguments: `*args` gathers extras into a tuple, `**kwargs` into a dict.

```
args_kwargs.py

1 def multiply(*kwargs):          # name is arbitrary! '*kwargs' packs extra pos:
2     p = 1
3     for i in kwargs: p *= i
4     return p
5 multiply(1,2,3,4,5)           # 120 - ANY count, no list needed
6 def display(**salman):         # '**salman' packs keyword args into a dict
7     for k, v in salman.items(): print(k, '->', v)
8 display(india='delhi', nepal='kathmandu')
```

Order matters : normal $\rightarrow$ `*args` $\rightarrow$ `**kwargs`. And args / kwargs are only CONVENTIONS - the `*` does the magic.

Q36. return - and the None trap?

```

return_none.py

1 L = [1,2,3]
2 print(L.append(4))      # None! append changes the list but returns nothing
3 print(L)                # [1,2,3,4]

```

Logic - No `return` → the function still returns `None`. `return` (1) exits immediately, (2) hands a value back: `z = f(x)`. A function can have many returns - one per branch.

Q37. Variable scope - global vs local?

```

scope.py

1 x = 5                      # GLOBAL
2 def g(y):
3     print(x)               # reads global x - allowed
4 def f(y):
5     x = 1                  # LOCAL x - shadows the global
6     x += 1; print(x)       # 2
7 def h(y):
8     x += 1                 # UnboundLocalError! (assigning makes it think local)

```

⚠ important - Assigning to a name ANYWHERE inside a function makes ALL uses of that name local. If you only meant to read the global → then don't assign it inside.

Logic - Numbers & strings pass BY VALUE - the function gets a copy. To get a result out, use `return`.

Q38. Nested functions & recursion?

```

nested.py

1 def f():
2     def g():
3         print('inside g')
4         f()          # recursion - f calls g calls f ...
5     g()
6     print('inside f')
7 f()                  # eventually RecursionError - python's safety limit

```

Logic - A function defined inside another is visible only there. Name lookup: **Local** → **Enclosing** → **Global** → **Built-in**. Every recursion needs a BASE CASE or it never ends.

Q39. Functions are first-class citizens?

Answer : A function is a **VALUE** like any other - it has an id, can be renamed, deleted, stored in containers, passed as an argument, or returned from another function.

```
first_class.py

1 type(square); id(square)  # it's an object with an address
2 x = square; x(3)          # alias & call -> 9
3 del square                # delete it
4 L = [1, 2, square]; L[-1](9) # store in a list
5 def f():
6     def x(a, b): return a+b
7     return x              # RETURN a function
8 f()(3, 4)                 # -> 7
9 def call_it(z): return z() # PASS a function as argument
```

Because functions flow like objects, we can build higher-order tools - map, filter, reduce - next! **Hint** :

Q40. Lambda - the anonymous function?

Answer : `lambda args: expression` - a function with NO name and only ONE expression (which is its return value).

```

lambda.py

1 a = lambda x: x**2      # assign a name to the anonymous fn
2 a(5)                   # 25
3 lambda s: 'a' in s      # boolean test
4 lambda x: 'even' if x%2==0 else 'odd' # ternary inside

```

<i>def</i>	<i>lambda</i>
has a name	anonymous (used on the spot)
many statements	ONE expression only
needs return	the expression IS the return
reusable everywhere	usually passed inline into HOFs

Q41. Higher Order Functions?

Answer : A function that takes a function and/or returns a function.

```

hof.py

1 def transform(f, L):      # f is a FUNCTION parameter
2     out = []
3     for i in L: out.append(f(i))
4     print(out)
5     transform(lambda x: x**3, [1,2,3,4,5])
6 # -> [1, 8, 27, 64, 125]  transform doesn't care WHICH function

```

Q42. map()?

Answer : `map(f, L)` applies `f` to EVERY item = `[f(x) for x in L]`.

map.py

```
1 list(map(lambda x: x**2, [1,2,3,4,5]))
2 # labels: ['odd','even','odd','even','odd']
3 list(map(lambda x: 'even' if x%2==0 else 'odd', [1,2,3,4,5]))
4 # extract one field from a list of dicts
5 list(map(lambda u: u['gender'], users))
```

⚠ important - map() is LAZY - it returns an iterator, not a list. Wrap with list() to force the work.

Q43. filter()?

Answer: filter(f, L) keeps only items where f returns True (it selects, it never transforms).

filter.py

```
1 list(filter(lambda x: x > 5, [3,4,5,6,7])) # [6,7]
2 list(filter(lambda x: x.startswith('a'), fruits)) # keep apple
```

map vs filter: map CHANGES every item, filter SELECTS some items. They compose - filter first, then map.

Q44. reduce()?

Answer: reduce keeps combining items into ONE final value - it "folds" the collection. Import it: from functools import reduce.

reduce.py

```
1 import functools
2 functools.reduce(lambda x, y: x+y, [1,2,3,4,5]) # 15
3 # (((1+2)+3)+4)+5 running total + next item
4 functools.reduce(lambda x, y: x if x > y else y, [23,11,45,10]) # 45 = max
```

Logic - The function always takes TWO args (accumulator, next item). Optional third arg = starting value. Use it when a collection must collapse into one result (sum, product, max, join).

TAGDA · Session 6 homework

- Write `is_even` yourself (mirror the docstring style).
- Make a `power(a, b)` that also works with zero args.
- Add a `multiply(*args)` that accepts any number of numbers.
- Square a list with `map`; keep only numbers > 5 with `filter`.
- Sum a list AND find its max - once with a loop, once with `reduce`.
- Write a function that RETURNS another function - and call it.

Pick the right container

Need...	Use	Because
an ordered, changing collection	list	mutable + ordered + fast to build
a fixed record / protection against edits	tuple	immutable, hashable, faster + smaller
unique items + instant "is it there?"	set	dedupes, $O(1)$ membership, math ops & - ^
key → value lookup	dict	access by key in $O(1)$; JSON-shaped nesting

Q45. The whole module in 8 lines?

Answer : **List** = flexible array (append/extend/insert, slicing, comprehension). **Tuple** = frozen list (unpacking, immutability). **Set** = unique + unordered (hash table). **Dict** = key→value map (get(), items(), comprehension). **Functions** = def + docstring + return, argument types, *args/**kwargs, scope, lambdas + map/filter/reduce.

Frequency cheatsheet - **list**: sorted/L.sort · count · index · copy · zip · traversal · comprehension
tuple: count · index · unpacking · swap · zip
set: add · update · remove/discard · pop · clear · | & - ^ · *_update methods · frozenset
dict: d[k] · get · items/keys/values · pop/popitem/del/clear/update · comprehension

The 30-second summary : lists = order + change · tuples = order + fixed · sets = unique + fast membership · dicts = keys to values. Choose the ONE that matches your data before you code.

FINAL · Before moving to Module 03 (Advanced)

- Re-run every error cell on purpose (TypeError, IndexError, KeyError, RecursionError) - errors are lessons.
- Solve all 7 list exercises + the 5 dict tasks from this module's Tasks folder.
- Try the 10 coding challenges in Interview_Questions before reading solutions.

-- end of Module 02 · next : 03_Python_Advanced --